

บทที่ 4

แนวคิดในการพัฒนาโปรแกรม

4.1 บทนำ

เบราว์เซอร์ที่สามารถโปรแกรมได้บนโทรศัพท์เคลื่อนที่ (Programmable Web Browser) คือ เว็บเบราว์เซอร์ที่สามารถทำหน้าที่หลักของเว็บเบราว์เซอร์ทั่วไป มีความสามารถในการแสดงผลภาษาเอ็กซ์เอ็มแอลได้ในระดับหนึ่ง สามารถทำงานได้บนโทรศัพท์เคลื่อนที่ที่สามารถรองรับ และมีโครงสร้างที่สนับสนุนในส่วนการทำงานของอินเทอร์พรีเตอร์ (ซึ่งส่วนอินเทอร์พรีเตอร์นี้ ไม่อยู่ในขอบเขตของปริณิงานี้)

เบราว์เซอร์ที่สามารถโปรแกรมได้บนโทรศัพท์เคลื่อนที่ทำให้ผู้ใช้งานโทรศัพท์เคลื่อนที่สามารถใช้บริการของเว็บไซต์ได้ผ่านเอกสารภาษา XHTML ได้ซึ่งจะทำให้ผู้ใช้งานสามารถเข้าถึงข้อมูลและบริการต่างๆได้ทุกสถานที่ทุกเวลา ซึ่งจะช่วยอำนวยความสะดวกเป็นอย่างมาก ซึ่งในการพัฒนานั้นผู้จัดทำโครงการจะต้องคำนึงถึงข้อจำกัดต่างๆของโทรศัพท์เคลื่อนที่ ความเร็วในการสื่อสารและ และวิธีการในการพัฒนาโปรแกรมด้วย

4.2 การเลือกโทรศัพท์เคลื่อนที่ในการพัฒนา

โทรศัพท์เคลื่อนที่ในปัจจุบันมีคุณสมบัติหลายประการที่แตกต่างกัน ซึ่งคุณสมบัติของโทรศัพท์เคลื่อนที่ที่จะต้องนำมาพิจารณาในการทำงานได้แก่

- หน่วยความจำ
- ระบบปฏิบัติการ
- ขนาดของการแสดงผล
- สีที่สามารถแสดง
- การเชื่อมต่อ
- ซอฟแวร์และภาษาที่รองรับ
- วิธีการรับอินพุตจากภายนอก
- การสนับสนุนจากผู้ผลิต

ข้อแตกต่างเพียงเล็กน้อยของคุณสมบัตินี้จะมีผลต่อการพัฒนาโครงการเป็นอย่างมาก ทั้งในเรื่องของการออกแบบ การประยุกต์วิธีในการพัฒนาและการติดต่อสื่อสารกับผู้ให้การสนับสนุน

ทางผู้จัดทำโครงการได้มีการติดต่อกับบริษัทอิริกสัน(ประเทศไทย) ผ่านท่านอาจารย์ที่ปรึกษา เพื่อขอยืมโทรศัพท์มือถือยี่ห้อ โซนี่อิริกสัน พี 800 (Sony Ericsson P800) เพื่อนำมาใช้ในการทดลอง นอกจากการ

อนุเคราะห์ที่ได้รับแล้ว โทรศัพท์มือถือรุ่นนี้ยังมีคุณสมบัติหลายอย่างที่เหมาะสมแก่การทำโครงการซึ่งได้แก่

- จอสีระบบสัมผัส (4096 สี)
- สนับสนุนการใช้งานภาษาจาวา (Java - personalJava and J2ME CLDC/MIDP)
- หน่วยความจำหลัก 12 เมกกะไบต์
- หน่วยความจำของเมมโมรีสต็อก (สามารถเพิ่มได้ตามต้องการ)
- รับส่งข้อมูลความเร็วสูง (HSCSD ได้ถึง 28.8 kbps)
- ระบบจีพีอาร์เอส GPRS (up to 53.6 kbps)
- การสนับสนุนจากบริษัท
- ระบบปฏิบัติการซิมเบียน (SymbianOS) เวอร์ชัน 7

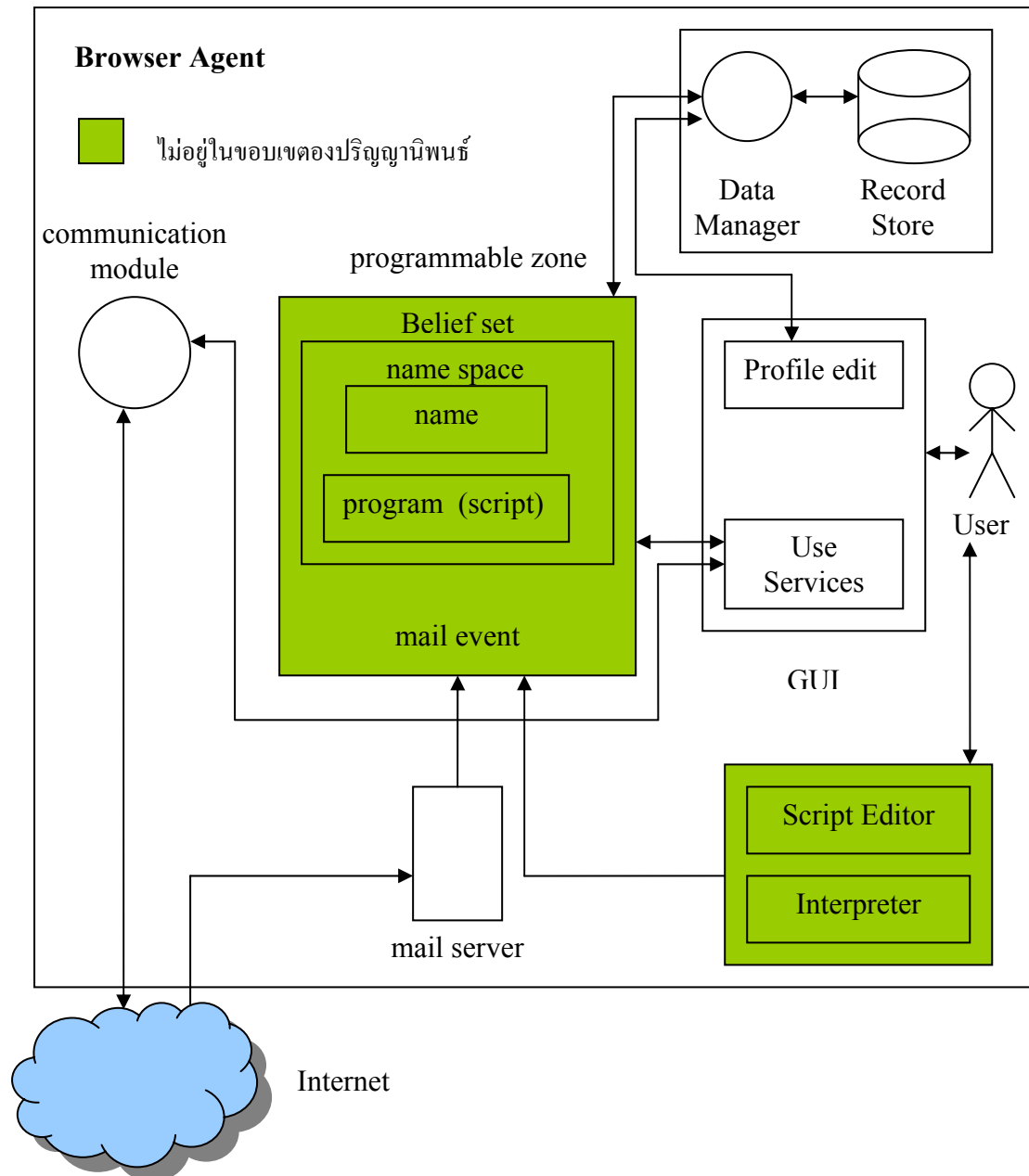
4.3 การวิเคราะห์การใช้งานของผู้ใช้

ในส่วนนี้ผู้จัดทำได้วิเคราะห์ถึงความต้องการใช้งานของผู้ใช้งานบราวเซอร์ที่สามารถโปรแกรมได้บนโทรศัพท์เคลื่อนที่ซึ่งจะมีลักษณะบางอย่างที่แตกต่างไปจากการใช้งานทั่วไป ได้แก่

- บราวเซอร์จะต้องประกอบด้วยฟังก์ชันการใช้งานที่บราวเซอร์ทั่วไปไม่มี ได้แก่
 - ส่วนที่ป้อนยูอาร์แอลของเว็บไซต์
 - ฟังก์ชันเดินหน้า(next)
 - ฟังก์ชันถอยหลัง(back)
 - ฟังก์ชันในจัดเก็บยูอาร์แอลที่ชอบเอาไว้ (add bookmark)
 - ฟังก์ชันในการแสดงยูอาร์แอลที่ชอบเอาไว้ (bookmark)
 - ฟังก์ชันในการแสดงหน้าแรก(home)
 - ฟังก์ชันในการเซตหน้าแรก (set home)
 - ฟังก์ชันโหลดเว็บเพจหน้านั้นซ้ำ (reload)
- ส่วนที่ใช้ในการปรับแต่งค่าต่างๆ
 - ส่วนในการเปลี่ยนสไตล์ของบราวเซอร์
 - ส่วนในการเปลี่ยนสีของบราวเซอร์
 - ส่วนเซตค่าทั้งหมดคืนสู่ค่าเริ่มต้น (default)
- ส่วนในการติดต่อกับอินเทอร์เน็ต
- ส่วนในการเชื่อมต่อกับเน็ตเวิร์ก
- ส่วนการรับส่งอีเมลล์
- ส่วนติดต่อกับผู้ใช้
 - รองรับอินพุตแบบสัมผัส (touch screen)
 - รองรับการอินพุตแบบกดปุ่ม (button)

ซึ่งการวิเคราะห์ข้อมูลอย่างละเอียดจะทำให้เราสามารถออกแบบโครงสร้างบราวเซอร์ที่สามารถโปรแกรมได้บนโทรศัพท์เคลื่อนที่ได้อย่างถูกต้องมากขึ้น

4.4 โครงสร้างของบราวเซอร์ที่สามารถโปรแกรมได้บนโทรศัพท์เคลื่อนที่



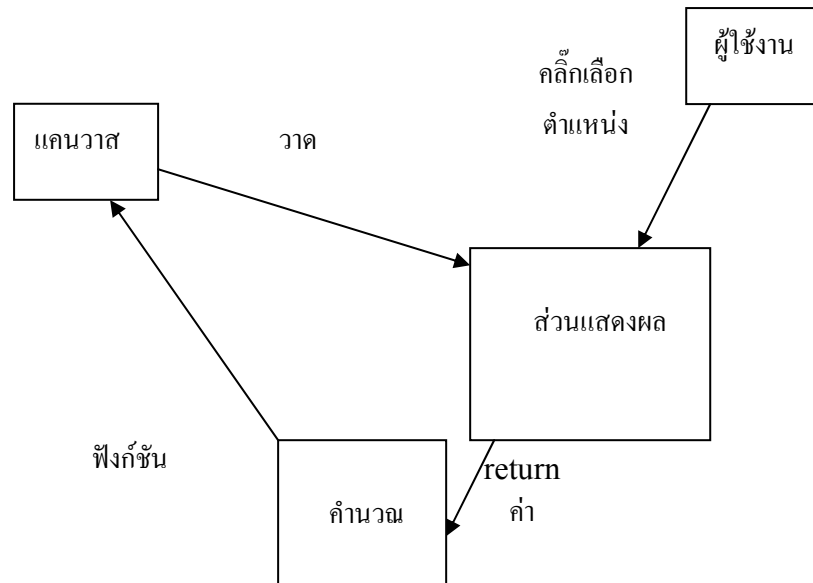
รูปที่ 4.1 โครงสร้างของบราวเซอร์ที่สามารถโปรแกรมได้บนโทรศัพท์เคลื่อนที่

โครงสร้างของบราวเซอร์ที่สามารถโปรแกรมได้บนโทรศัพท์เคลื่อนที่ประกอบด้วย

1. ส่วนติดต่อกับผู้ใช้งาน(GUI:Graphics User Interface) ซึ่งส่วนนี้จะเป็นส่วนที่ใช้ป้อนคำสั่ง สกริปต์ให้แก่ตัวอินเทอร์พรีเตอร์(interpreter) โดยผ่านส่วนของการป้อนสกริปต์โดยตรง (Script Editor) การสั่งงานผ่านลิสต์อีเวนต์ (Event – action list) นอกจากนั้นยังเป็นส่วนของการใช้งานบราวเซอร์ทั่วไป(use services) ส่วนการปรับแต่งค่า(Profile Editor) ซึ่งจะนำค่ามารองรับคำสั่งและนำค่าต่างๆมาแสดงผลต่อผู้ใช้งาน (user)
2. การจัดการหน่วยความจำ ซึ่งจะประกอบด้วยส่วนจัดการข้อมูล (data manager) เช่น อ่านค่า เขียนค่า ลบค่า เช็ดค่า และส่วนจัดเก็บข้อมูล (Record set) คือ ส่วนที่จัดเก็บข้อมูลอย่างถาวรไว้ในหน่วยความจำ ซึ่งจะเก็บค่าต่างๆไว้ ได้แก่ ข้อมูลของ user ข้อมูลของระบบ
3. ส่วนที่เกี่ยวข้องกับการติดต่อสื่อสาร (Communication module) จะเป็นส่วนที่จัดการเกี่ยวกับการสื่อสารระหว่างบราวเซอร์กับอินเทอร์เน็ต ซึ่งจะมีการทำงานในส่วนของการเชื่อมต่อใช้ในการดูเอกสารอิเล็กทรอนิกส์ที่เอ็มแอล โดยกรอกยูอาร์แอลให้กับเว็บบราวเซอร์จากนั้นเว็บบราวเซอร์จะร้องขอข้อมูลมาให้และแสดงผลที่หน้าจอ
4. บิเลฟเซต (Belief set) คือ ส่วนข้อมูล หรือความรู้ของเอเจนต์ อันได้แก่ เนม และสกริปต์ซึ่งจะมีชื่ออยู่ในเนมสเปซของเอเจนต์ และอีเวนต์ – แอ็กชันที่เมื่อไรก็ตามที่อีเวนต์เกิดขึ้นแล้ววเอเจนต์จะทำงานตามแอ็กชันนั้น บิเลฟเซตประกอบด้วย 3 ส่วน คือ
 - a. เนม (Name) - คือชื่อที่ผู้ใช้อ้างถึงข้อมูลต่างๆ ภายในเอเจนต์ซึ่งเนมในมุมมองของผู้ใช้ก็คือตัวแปรที่ใช้อ้างถึงข้อมูลต่าง ๆ นั้นเอง โดยที่ผู้ใช้สามารถเพิ่มเติมเนมเข้าไปเป็นฐานความรู้ให้แก่เอเจนต์ได้
 - b. สกริปต์ (Script) – คือส่วนของโปรแกรมที่เอเจนต์มีอยู่ทั้งในรูปแบบของฟังก์ชันและคลาส ผู้ใช้สามารถเพิ่มเติมโปรแกรมเข้าไปเป็นฐานความรู้ให้แก่เอเจนต์ได้
 - c. อีเวนต์ – แอ็กชัน (Event - action) – คือฐานความรู้ในอีกรูปแบบหนึ่งมีความหมายว่าเมื่ออีเวนต์เกิดขึ้นแล้วจะให้ทำแอ็กชันอะไร โดยมันจะถูกเก็บอยู่ในฐานความรู้ของเอเจนต์และถูกตรวจสอบอีเวนต์อยู่เสมอจนเมื่ออีเวนต์เป็นจริงแล้วก็จะทำแอ็กชัน และรายงานผลของแอ็กชันว่าเป็นเช่นไร

4.5 ขั้นตอนการสร้างส่วนติดต่อกับผู้ใช้งานแบบระบบสัมผัส

สำหรับโทรศัพท์เคลื่อนที่รุ่น พี800 นี้มีลักษณะพิเศษคือ สามารถรับอินพุตจากผู้ใช้งานผ่านวิธีการสัมผัสได้ ซึ่งการออกแบบโปรแกรมโดยใช้ภาษาจาวานั้นจะต้องใช้วิธีที่เรียกว่า การแคนวาส (canvas) ซึ่งเป็นวิธีการในการวาดรูปลงไปโดยกำหนดเป็นจุดพิกเซล วิธีนี้เท่านั้นที่จะทำให้โทรศัพท์เคลื่อนที่สามารถรองรับการระบบสัมผัสได้



รูปที่ 4.2 การทำงานของส่วนติดต่อกับผู้ใช้

ตัวอย่างขั้นตอนการสร้างเมนูของบราวเซอร์ให้รองรับการทำงานแบบระบบสัมผัส

- ในขั้นแรกในส่วนของโปรแกรมจะต้องสามารถใช้งานแคนวาสได้ เช่น ถ้าต้องการให้คลาสชื่อ WEB สามารถใช้งานแคนวาสจะต้องเขียนดังต่อไปนี้

```
import javax.microedition.lcdui.*;
```

```
class Web extends Canvas
```

```
{
```

```
}
```

- ใช้เมธอด (method) ของแคนวาสที่ชื่อว่า `protected void pointerPressed(int x,int y)` ซึ่งจะทำให้สามารถใช้งานระบบสัมผัสได้
- สร้างเมธอดที่จะทำค่า ณ จุดพิกเซลที่มีการสัมผัสส่งไปเพื่อประมวลผลว่า เมื่อมีการสัมผัสแล้วจะให้มีการทำงานอย่างไรต่อไป ในที่นี้จะสร้างเมธอด `SelectMenu` เพื่อใช้คำนวณว่า เลือกเมนูตำแหน่งที่เท่าไร และจะรีเทิร์นค่าตำแหน่งนั้นออกมาเพื่อให้ midlet แสดงผล

```

public int SelectedMenu(int x,int y)
{
    int position=0;

    if((x>0*menu_w)&&(x<1*menu_w)&&(y>0*menu_w)&&(y<1*menu_h))
        position= 1;
    else
    if ((x>1*menu_w)&&(x<2*menu_w)&&(y>0*menu_w)&&(y<1*menu_h))
        position= 2;
    else
    if ((x>2*menu_w)&&(x<3*menu_w)&&(y>0*menu_w)&&(y<1*menu_h))
        position= 3;
    .
    return position;
}

```

- จากนั้นนำค่าที่รีเทิร์นกลับมาแสดงผล รวมเป็น

```

protected void pointerPressed(int x,int y)
{
    int menu_num=menu.SelectedMenu(px,py);
    switch (menu_num)
    {
        case 1:midlet.ShowWeb();
        case 2:midlet.ShowScript();break;
        case 3:midlet.ShowMail();break;
        case 4:midlet.ShowFTP();break;
        case 5:midlet.ShowUser();break;
        case 6:midlet.ShowSetting();break;
        case 7:midlet.ShowInfo();break;
    }
}

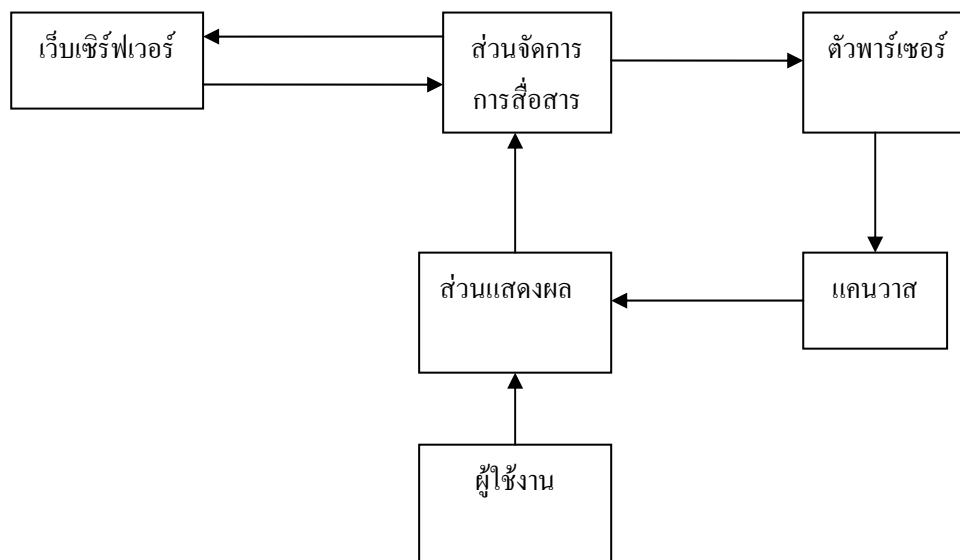
```

4.6 การแสดงผลเอกสารเอ็กซ์เอ็มแอล(XHTML)

เนื่องจากภาษาเอ็กซ์เอ็มแอลเป็นภาษาที่มีลักษณะเดียวกับกับภาษาเอ็กซ์เอ็มแอล ดังนั้นการทำให้หน้าแรกของภาษาเอ็กซ์เอ็มแอลมาแสดงผลนั้นต้องการอาศัยการวิเคราะห์โครงสร้างเอกสารภาษา

เอ็กซ์เอ็มแอล ชุดของแบบแผนในการทำเครื่องหมายส่วนต่างๆ ของข้อความเพื่อที่ว่าเมื่อเข้าถึงโปรแกรมที่เรียกว่า “พาร์เซอร์” (parser) ซึ่งเป็นโปรแกรมที่แบ่งข้อมูลส่วนใหญ่ให้เล็กลงเพื่อง่ายต่อการแปลความหมาย ส่วนต่างๆ เหล่านั้นจะปรากฏขึ้นในรูปแบบต่างๆ กัน เพราะฉะนั้นการวิเคราะห์โครงสร้างของภาษาเอ็กซ์เอ็มแอลจึงต้องใช้สิ่งที่เรียกว่า เอ็กซ์เอ็มแอลพาร์เซอร์ (XML parser) ซึ่งจะช่วยให้เราได้โครงสร้างออกมาแสดงผลต่อไป

ซึ่งในปัจจุบันยังไม่มีตัวเอ็กซ์เอ็มแอลพาร์เซอร์ ที่สามารถนำภาษาเอ็กซ์เอ็มแอลมาแสดงผลได้อย่างสมบูรณ์เต็มที่ นอกจากนั้นยังไม่รองรับการทำงานบนโทรศัพท์เคลื่อนที่ที่ใช้ภาษาจาวาไม่ไคโรอีดิชัน ทางผู้จัดทำจึงได้สร้างรูปแบบของการพาร์เซอร์เอกสารขึ้นเอง ซึ่งทำให้ง่ายต่อการนำแท็กในภาษาเอ็กซ์เอ็มแอลมาแสดงผลมากขึ้น



รูปที่ 4.3 โครงสร้างการทำงานของพาร์เซอร์

แนวคิดในการทำงานของการแสดงผลภาษาเอ็กซ์เอ็มแอล

1. จากรูปผู้ใช้งานทำการร้องขอเอกสารเอ็กซ์เอ็มแอลผ่านการกรอกยูอาร์แอล เช่น <http://127.0.0.1/projext/test.html> ผ่านส่วนติดต่อกับผู้ใช้
2. ส่วนจัดการการสื่อสารจะร้องขอเอกสารไปยังเว็บเซิร์ฟเวอร์
3. เว็บเซิร์ฟเวอร์ตอบรับเอกสารเอ็กซ์เอ็มแอลกลับมายังส่วนจัดการการสื่อสาร
4. ตัวพาร์ซernำเอกสารเอ็กซ์เอ็มแอลมาวิเคราะห์โครงสร้างและคำนวณการแสดงผลสำหรับแท็กต่างๆ
5. ตัวพาร์เซอร์ส่งค่าตัวแปรที่จำเป็นสำหรับการแสดงผลยังส่วนของการแคนวาส เพื่อใช้ในการคำนวณการวาด

6. แผนวาสนาที่แตกต่างกันมาแสดงผลอย่างถูกต้องให้กับผู้ใช้งาน

4.7 ขั้นตอนในการวิเคราะห์รูปแบบและวิธีการในการแสดงผลเอกสาร XHTML

4.7.1 เอ็กเอ็มแอลพาร์เซอร์

ในการที่พาร์เซอร์จะนำเอกสาร XML มาทำวิเคราะห์โครงสร้างนั้น ในปัจจุบันมีมาตรฐานหรือวิธีการหลักๆอยู่ 2 อย่างได้แก่ DOM (Document Object Model) และ SAX (Simple API for XML) ซึ่งทั้งสองวิธีต่างก็มีข้อดีและข้อเสียอยู่ในตัว ดังนั้นจึงได้มีบางคนพยายามคิดวิธีการอื่นๆ ขึ้นมา แต่อย่างไรก็ตาม ก็ยังคงยึดแนวทางของ DOM และ SAX เป็นหลักอยู่นั่นเอง

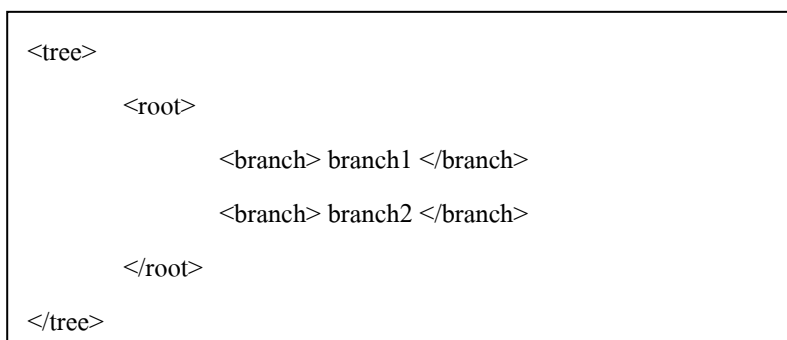
การมองโครงสร้างเอกสารเป็นทรีด้วย DOM

XML DOM คือแอปพลิเคชันโปรแกรมมิ่งอินเทอร์เฟซ (Application Programming Interface, API) สำหรับเอกสาร XML ซึ่งจะกำหนดว่าเอกสาร XML จะมีการเข้าถึงและการโยกย้ายรวมไปถึงการจัดการต่างๆ ได้อย่างไรบ้าง DOM เป็นมาตรฐานของ W3C โดยมีเป้าหมายคือต้องการสร้างอินเทอร์เฟซที่เป็นมาตรฐานและจะต้องใช้กับแอปพลิเคชันได้หลากหลาย ซึ่ง DOM ก็สามารถใช้ได้กับภาษาโปรแกรมมิ่งหรือระบบปฏิบัติการใดก็ได้ ด้วยวิธีการแบบ DOM โปรแกรมเมอร์จะสามารถสร้างเอกสาร XML ขึ้นมาเข้าไปในโครงสร้างระดับต่างๆ ได้ รวมทั้งเพิ่ม ลบ และแก้ไขอีลีเมนต์ต่างๆ ของเอกสารได้

มุมมองของ DOM

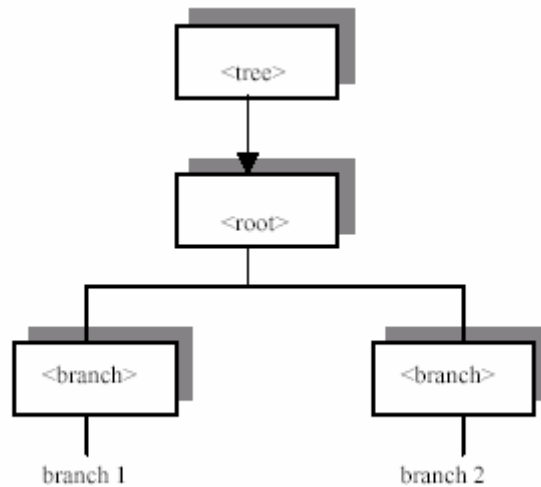
การทำงานของ DOM จะเริ่มต้นเมื่อโปรแกรมหรือแอปพลิเคชันไปเรียก (call) พาร์สเซอร์ให้ไปโหลดเอกสาร XML เข้าสู่หน่วยความจำของคอมพิวเตอร์ เมื่อเอกสารถูกโหลดเสร็จเรียบร้อยแล้ว DOM ก็จะทำให้โปรแกรมสามารถดึงข้อมูลมาใช้งานหรือดำเนินการต่างๆ กับข้อมูลได้ DOM จะมีมุมมองไปยังเอกสาร XML เป็นโครงสร้างแบบต้นไม้หรือทรี (tree) ระดับบนสุด ของทรีเราจะเรียกว่า “documentElement” ซึ่งอีลีเมนต์ดังกล่าวจะแตกสาขาออกไปเป็นอีลีเมนต์ย่อยๆ ตั้ง

แต่ 1 อีลีเมนต์ขึ้นไป โดยอีลีเมนต์ย่อยที่แตกออกจาก documentElement เราจะเรียกว่า “โหนดลูก” (childNodes)



รูปที่ 4.4 แสดงโค้ดตัวอย่างภาษา XML

DOM จะมองเอกสารเป็นทรีดังนี้คือ



รูปที่ 4.5 มุมมองของ แท็ก XML ในรูปแบบของ DOM

จุดเด่นของ DOM

- โปรแกรมจะพาร์สเอกสารเพียงครั้งเดียว เมื่อ DOM ได้สร้างทรีขึ้นมาแล้ว ทรีนั้นก็จะอยู่ในหน่วยความจำตลอดจนกว่าเราจะสั่งยกเลิก และในขณะที่ทรีอยู่ในหน่วยความจำ เราก็สามารถเข้าถึงข้อมูลที่อยู่ภายในทรีได้อย่างรวดเร็ว
- ข้อมูลที่สร้างเป็นทรีจะทำให้ข้อมูลเกิดความสัมพันธ์ ทำให้การเรียกใช้งานทำได้ง่าย

จุดด้อยของ DOM

- ในกรณีที่เอกสารมีขนาดใหญ่และซับซ้อน การสร้างทรีจะทำได้ช้า และกินหน่วยความจำมาก
- ผู้พัฒนาโปรแกรมจำเป็นต้องเข้าใจถึงโครงสร้างโดยรวมของเอกสารเป็นอย่างดี จึงจะสามารถเขียนโปรแกรมโดยใช้ DOM ได้ กล่าวคือในส่วนของการเขียนโค้ดด้วยวิธีการของ DOM นั้นจะค่อนข้างยาวและยุ่งยากอยู่พอสมควร

มองโครงสร้างเอกสารเป็นเส้นโครงด้วย SAX

SAX (simple API for XML) เป็นแอปพลิเคชัน โปรแกรมมิ่งอินเทอร์เฟซ (API) ที่มีลักษณะการทำงานก็ต่อเมื่อมีเหตุการณ์ (เช่น การร้องขอข้อมูล) ใดๆ เกิดขึ้นหรือเรียกโมเดลการทำงานแบบบน event-based model กล่าวคือ SAX จะไม่มีการสร้างภาพรวมของเอกสารไว้ก่อนเลยว่าเอกสารมีโครงสร้างเป็นเช่นไร เมื่อแอปพลิเคชันมีการร้องขอมาจึงจะรายงานหรือทำงานตามเหตุการณ์ของการทำพาร์ส (parsing) ไปยังแอปพลิเคชันดังกล่าว หลักการทำงานของ SAX จะมองเพียง “จุดเริ่มต้น” กับ “จุดสิ้นสุด” ของเอกสารและของอากิวต์ เมนต์เท่านั้น และจะสนใจเฉพาะสิ่งที่ต้องการเท่านั้น เมื่อเปรียบเทียบกับ

วิธีการของ DOM ซึ่งนำข้อมูลทั้งหมดมาสร้างโครงสร้างข้อมูลทั้งหมดมาสร้างโครงสร้างข้อมูลเป็นทรี (tree) ทำให้ DOM มีข้อจำกัดในเรื่องหน่วยความจำซึ่งจะต้องใช้หน่วยความจำมากหากทรีที่สร้างมีขนาดใหญ่หลายๆ ส่วน SAX จะเข้าถึง ข้อมูลได้ง่ายกว่า การเขียนโปรแกรมทำได้ง่ายกว่าเพราะมองข้อมูลในลักษณะของเส้นตรงและสามารถพาร์สเอกสารที่มีขนาดใหญ่มากกว่าจำนวนหน่วยความจำที่เรียกได้ ดังนั้นโปรแกรมระบบฐานข้อมูล ใหญ่ๆหลายบริษัทจึงนิยมที่จะใช้วิธีนี้กันเพื่อให้เกิดความเข้าใจในการทำงานของ SAX มากขึ้น เราจะมาพิจารณาจากตัวอย่างเอกสาร XML ง่ายๆดังนี้

```
<doc>
  <element1>Value 1</element>
  <element2>Value 2</element2>
</doc>
```

รูปที่ 4.6 แบบตัวอย่างของเอกสาร XML

SAX จะมองเอกสารข้างต้นไปที่ละบรรทัดๆ ในลักษณะเหตุการณ์เชิงเส้น (linear events) ดังนี้

Start document

Start element : doc

Start element : element1

Characters : Value1

end element : element1

พาร์สเอกสารตั้งแต่บรรทัดแรกจนถึงบรรทัด

Start element : element2

สุดท้ายแบบเส้นตรง

Characters : Value2

end element : element2

end element : doc

end document

ตัวอย่างการพาร์สแบบ SAX ข้างต้นเป็นการพาร์สทั้งเอกสาร ในทางปฏิบัติแล้วเรามักไม่ได้มีการพาร์สเอกสารทั้งหมด เมื่อเราได้ข้อมูลที่ต้องการแล้วก็จะหยุด วิธีการพาร์สแบบ SAX จึงทำงานได้อย่างรวดเร็วและเขียนโปรแกรมได้ง่ายมาก

จุดเด่นของ SAX

- เข้าถึงข้อมูลได้ง่าย และรวดเร็ว
- เขียนโปรแกรมอิมพลิเมนต์ได้ง่าย เนื่องจากการเข้าถึงข้อมูลเป็นลักษณะเส้นตรง
- ใช้หน่วยความจำน้อย สามารถเข้าถึงข้อมูลหรือเอกสารที่มีขนาดใหญ่กว่าหน่วยความจำที่มีของเครื่องได้

จุดด้อยของ SAX

- ไม่สามารถบอกความสัมพันธ์ของข้อมูลได้
- เป็นวิธีการที่ไม่ค่อยมีประสิทธิภาพหากเป็นการเข้าถึงข้อมูลเดิมบ่อยๆ

เปรียบเทียบการทำงานระหว่าง DOM และ SAX

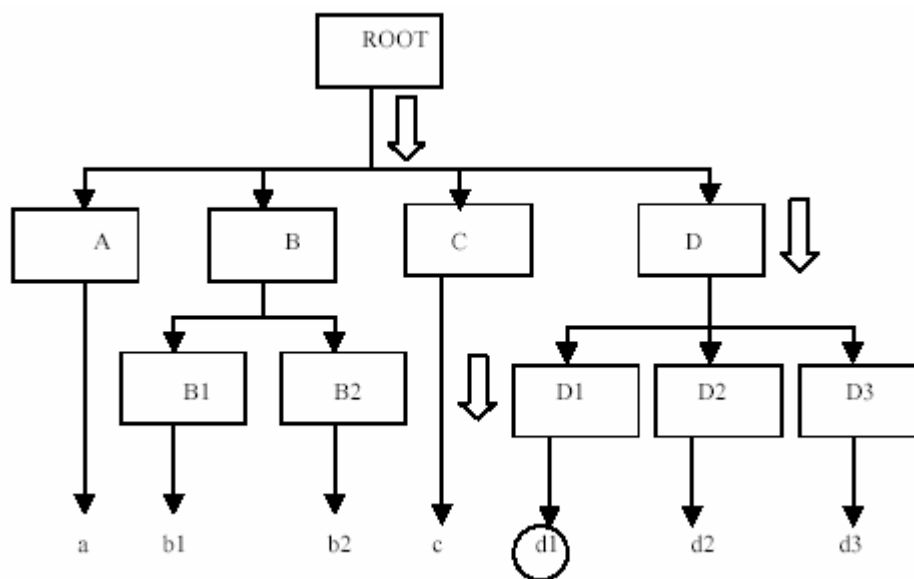
ทั้ง DOM และ SAX ต่างมีจุดเด่นด้อยอยู่ในตัว การใช้งานจึงขึ้นอยู่กับดุลพินิจของผู้พัฒนาว่า แอปพลิเคชันที่ใช้เอกสาร XML นั้นมีความเหมาะสมกับวิธีการใดมากที่สุด

ลองเปรียบเทียบการทำงานระหว่าง DOM กับ SAX จากเอกสารชุดเดียวกัน โดยสมมติว่าเรามีเอกสาร XML ที่มีรายละเอียดดังนี้

```
<ROOT>
  <A>a</A>
  <B>
    <B1>b1</B1>
    <B2>b2</B2>
  </B>
  <C>c</C>
  <D>
    <D1>d1</D1>
    <D2>d2</D2>
    <D3>d1</D3>
  </D>
</ROOT>
```

จากเอกสาร XML ข้างต้น ถ้าเป็นวิธีการของ DOM จะมองเอกสารเป็นทรีที่เกิดจากการประกอบกันของโหนดต่างๆ และจะมองอีลิเมนต์แต่ละตัวเป็นออบเจกต์ แล้วเมื่อเราต้องการเข้าถึงข้อมูลใด DOM ก็จะใช้วิธีเข้าไปยังโหนดต่างๆ ในทรีเพื่อหาข้อมูลที่ต้องการ เช่น เราต้องการหาค่าของอีลิเมนต์ D1 ว่ามีค่า

เท่าใด ก็จะมีลักษณะการทำงานดังรูป



รูปที่ 4.7 แสดงการพาร์เอกสารแบบ DOM

ส่วน SAX จะไม่มีการสร้างมุมมองใดๆ ของเอกสารได้เลย การทำงานของ SAX จะเริ่มต้นก็ต่อเมื่อเกิดเหตุการณ์ใดๆ ขึ้นเท่านั้น เช่น เราต้องการรู้ว่าอีลิเมนต์ D1 มีค่าเท่าไร SAX ก็จะเริ่มอ่านเอกสารดังนี้

Start document

Start element : root

Start element : A

Characters : A

End element : A

Start element : B

Start element : B1

Characters : b1

End element : B1

Start element : B2

Character b2

End element: B2

End element : B

Start element : C

Character c

End element : C

Start element : D

Start element : D1

Characters : d1

End element : D1

.
.
 .

ส่วนที่เหลือนี้พาร์เซอร์จะไม่ทำการประมวลผล

4.7.2 รูปแบบของพาร์เซอร์ที่เลือกใช้งาน

ทั้ง DOM และ SAX ต่างมีจุดเด่นจุดด้อยไปคนละแบบ ในทางปฏิบัติจึงขึ้นอยู่กับลักษณะการใช้งานว่าจะเหมาะสมกับวิธีการพาร์สแบบใด เช่น หากให้ความสำคัญกับความสัมพันธ์ของข้อมูลก็ควรใช้ DOM แต่ถ้าต้องการวิธีการที่ทำให้เขียนโปรแกรมได้ง่ายและไม่เปลืองหน่วยความจำในการประมวลผลก็ควรใช้ SAX เป็นต้น ซึ่งในปัจจุบันก็มีอยู่หลายระบบงานที่ใช้ทั้งสองวิธีผสมผสานกัน

ซึ่งในโครงการนี้ได้ใช้หลักการของ DOM มาใช้ในการ พาร์สเอกสาร ซึ่ง ได้ทำการพัฒนาพาร์เซอร์ (parser) ขึ้นมาเองเนื่องจากว่า ภาษาจาวาทูเอมอินั้น เป็นภาษาที่ยังใหม่อยู่ และมีข้อจำกัดอย่างมาก ซึ่งทางไลบรารีของภาษาจาวานั้น ไม่มีตัวแปลเอกสาร XML ให้ดังนั้นทางผู้ทำโครงการจึงต้องทำตัวพาร์สเอกสาร XML ขึ้นมาเอง และ เนื่องจากว่าโครงการนี้เกี่ยวกับ บราวเซอร์ ดังนั้นเราจึงได้ทำการพาร์สเอกสาร XHTML ซึ่งเป็นภาษาลูกของ XML

4.7.3 ขั้นตอนการพัฒนาการพาร์สเอกสาร

จากที่กล่าวมาว่า ตัวแปลเอกสารของโครงการนี้ใช้หลักการเดียวกับ DOM คือจะอ่านเอกสารทั้งหมดแบบ แผนภูมิต้นไม้ และจะเก็บค่าไว้ในหน่วยความจำ จากนั้นก็นำแท็กที่แปลออกมา มาแสดงผลต่อไป

- ขั้นตอนการร้องขอเพื่อดูเอกสาร XHTML

เมื่อ ผู้ใช้ได้ทำการป้อน URL และกดตกลงเป็นที่เรียบร้อยแล้ว บราวเซอร์ก็จะส่ง การร้องขอไปยัง เซิร์ฟเวอร์ ซึ่งก็จะได้รับ เอกสาร XHTML เป็น สตรีมเข้ามา ซึ่งทาง เราก็จะทำการวนลูปเพื่อนำสตรีมเหล่านั้น มาเก็บไว้ในตัวแปรสตริง ก่อนที่จะนำไปพาร์สต่อไป ซึ่งได้ใช้ เมธอด ดึงซอร์สโค้ดดังต่อไปนี้

```
http = (HttpConnection) Connector.open(url);
http.setRequestMethod(HttpConnection.GET);
http.setRequestProperty("User-Agent", "Profile/MIDP-1.0 Configuration/CLDC-1.0");
if (http.getResponseCode() == HttpConnection.HTTP_OK)
{

String str;

iStrm = http.openInputStream();
int length = (int) http.getLength();
if (length != -1)
{
    // Read data in one chunk
    byte serverData[] = new byte[length];
    iStrm.read(serverData);
    str = new String(serverData);
}
else // Length not available...
{
    ByteArrayOutputStream bStrm = new ByteArrayOutputStream();

    // Read data one character at a time
    int ch;
    while ((ch = iStrm.read()) != -1)
        bStrm.write(ch);

    str = new String(bStrm.toByteArray());
    bStrm.close();
}
```

เมื่อเราได้รับเอกสาร XHTML มาเป็นที่เรียบร้อยแล้ว เราก็จะส่งไปทำการพาร์สต่อดังนี้

- ขั้นตอนการพาร์สเอกสาร XHTML

ก่อนอื่นเราต้องทำความเข้าใจกันก่อนว่า เราได้นำเอกสารเก็บไว้ในตัวแปลสตริงเป็นที่เรียบร้อยแล้ว ดังนั้นต่อไปก็เป็นการแปลแท็กต่าง ๆ ในสตริงที่รับมา ซึ่งการแปลแท็กต่าง ๆ ทั้งหมดนั้นใช้หลักการที่เป็นแบบเดียวกันโดยหลักการทั้งหมดมีดังนี้

1. ทำการหาแท็กเริ่มต้น หรือหา แท็กเปิด
2. หากเจอแท็กเปิด แล้วก็ให้ทำการตรวจสอบดูชื่อของแท็ก

ดังตัวอย่างโค้ดต่อไปนี้

```
for (i=0;i<str.length()-1;i++)
{
    // ตรวจสอบว่าเป็นแท็กเปิดหรือไม่
    if ((str.charAt(i)=='<')&&(open_tag==false)&&(close_tag==true)
        {open_tag=true;close_tag=false;}
    // หากเป็นแท็กเปิดแล้ว เราก็จะทำการตรวจสอบดูว่าเป็นแท็กชื่ออะไร
    else if (open_tag)//check tag
    {
        switch(str.charAt(i))
        {
            case'?:'{}
            case'!':{}
            ...
            ...
            default;;break;
        }
    }
}
```

3. การตรวจสอบเพื่อดูว่าแท็กที่รับมานั้นเป็นแท็กชื่ออะไร เราจะใช้หลักการดังนี้

ดูว่าตัวแรกเป็นตัวอะไร ซึ่งหากเป็น แท็กที่ตัวแรกไม่เหมือนแท็กอื่น ๆ เลย และเป็นแท็กที่รู้จะเราก็สามารถตอบได้ว่าเป็นแท็กอะไร เช่น แท็ก <var> ซึ่งเป็นแท็กที่บอกว่าเป็น ตัวแปร (a variable) แต่จะมี

หลาย ๆ แท็กที่ตัวแรกเหมือนกัน เช่น , <body>, <big> เป็นต้นซึ่งในกรณีนี้เราต้องไปทำยังขั้นตอนถัดไป

ทำการตรวจสอบตัวอักษรลำดับถัดมาว่าเป็นตัวอะไร และหากมีแท็กที่ตัวอักษรลำดับต่อมานั้น ซ้ำกันอีกเราก็จะตรวจตัวอักษรตัวถัดมาอีกเรื่อย ๆ จนไม่มีตัวอักษรที่ซ้ำกันและ สามารถบอกได้ว่าแท็กนั้นเป็นแท็กอะไร ดังโค้ดต่อไปนี้

```
//หากเป็นแท็กเปิด ก็ให้ดูต่อไปว่าเป็นแท็กอะไร
if (open_tag)
{
    switch (str.charAt(i))
    {
        //หากตัวอักษรตัวแรกเป็น ตัวb ก็ให้ดู ตัวอักษรตัวถัดไป
        case'b':{
            if ((str.charAt(i+1)=='>')&&(b_tag==false))
            // หากตัวอักษรตัวที่ สองเป็น ">" เราสามารถทราบได้เลยว่า แท็กนี้ต้องเป็น แท็ก <b> //
            //แน่นอน
            {
                // ส่วนนี้จะใส่วิธีการ จัดการ และส่งค่าของแท็กนี้ไป ซึ่งจะกล่าวอีกครั้งใน
                //ภายหลัง
                break;
            }
            // หากตัวอักษรตัวที่ สองเป็น "o" เราสามารถทราบได้เลยว่า แท็กนี้ต้องเป็น แท็ก
            //<body> แน่แน่นอน
            if ((str.charAt(i+1)=='o')&&(body_tag==false))
            {
                // ส่วนนี้จะใส่วิธีการจัดการและส่งค่าของแท็กนี้ไป ซึ่งภายหลัง
                break;
            }
        }
```

```

// หากตัวอักษรตัวที่ สองเป็น “i” เราสามารถทราบได้เลยว่า แท็กนี้คือ แท็ก <big> แน่แน่นอน
{
// ตรงนี้จะใส่วิธีการ จัดการ และส่งค่าของแท็กนี้ไป ซึ่งจะกล่าวอีกครั้งในภาคหลัง
break;
}
// หากตัวอักษรตัวที่ สองเป็น “r” เราสามารถทราบได้เลยว่า แท็กคือ แท็ก <br/>
if (((str.charAt(i+1)=='r')&&(str.charAt(i+2)=='/')&&(str.charAt(i+3)=='>'))
|| ((str.charAt(i+1)=='r')&&(str.charAt(i+3)=='/')&&(str.charAt(i+4)=='>'))
{
// ส่วนนี้จะใส่วิธีการ จัดการ และส่งค่าของแท็กนี้ไป ซึ่งจะกล่าวอีกครั้งในภาคหลัง
}
break;
}
...
...
}

```

4. หากว่าแท็กใดมีแอตทริบิว เช่นแท็ก เราก็จะส่งค่าแอตทริบิวทั้งหมดไปให้ส่วนแสดงผลเลย และ เราจะแยกแอตทริบิวทั้งหมดนั้นอีกครั้งที่ส่วนแสดงผล ให้เหลือเฉพาะ แอตทริบิวที่จำเป็น และ เครื่องโทรศัพท์เคลื่อนที่สามารถแสดงผลได้นั้น มาทำการคำนวณเพื่อใช้ในการประมวลผลต่อไป ตัวอย่างดังนี้

```

//หากอักษรตัวแรกเป็น m ดังนั้นเราสามารถรู้ได้เลยว่าแท็กที่เจอนี้ต้องเป็นแท็ก m
if (str.charAt(i+1)=='m'
{
// เป็นการแท็กให้หน่วยแสดงผล
output_tag[count]="<img>";

// เป็นการส่งค่าแอตทริบิวให้หน่วยแสดงผล
for(j=i+3;(str.charAt(j)!='>');j++){

```

```

        output_tag_attr[count]=str.substring(i+3,j);
        for(j=i;(str.charAt(j)!='>');j++) {}
        for(k=j;(str.charAt(k)!='<');k++) {}
        output_value[count++]=str.substring(j+1,k);
    }

    for(j=i;(str.charAt(j)!='>');j++) {}
    for(k=j;(str.charAt(k)!='<');k++) {}

    // เป็นการส่งค่าลำดับแท็กให้หน่วยแสดงผล
    output_value[count++]=str.substring(j+1,k);
    NumberOfAttribute=NumberOfAttribute+1;
    open_tag=false;
    break;
}

```

ส่วนของการแสดงผลโดยส่วนใหญ่นั้นในแต่ละแท็กจะมีรูปแบบและการทำงานในส่วนแสดงผลคล้ายกัน ดังนั้นจึงขอยกตัวอย่างส่วนแสดงผลดังนี้

ตัวอย่างซอร์สโค้ดของการจัดการแท็ก <h1> เพื่อมาแสดงผล

แท็ก <H1> จะเป็นแท็กที่มีรูปแบบในการเปลี่ยนแปลงขนาดของตัวอักษร ซึ่งสามารถจัดการได้ในรูปแบบของการทำตัวอักษรนั้นมาเปลี่ยนขนาดโดยจาวา

```

if (parserz.output_tag[Current_tag_value_attribute]=="<h1>")
{
    //Flag for position of begin and end of link

    //เราจะทำการเปลี่ยนขนาดตัวอักษรของข้อความหลัง แท็ก <h1> ตรงนี้ก่อนที่จะ
    //แสดงผล
    g.setFont(Font.getFont(Font.FACE_SYSTEM,Font.STYLE_BOLD,Font.SIZE_LARGE));

    // นำข้อความเข้าฟังก์ชันแสดงผล
    Display(parserz.output_value[Current_tag_value_attribute],g,RGB)
    ;

    g.setFont(Font.getFont(Font.FACE_SYSTEM,Font.STYLE_PLAIN,Font.SIZE_MEDIUM));

    //เราจะทำการเปลี่ยนขนาดตัวอักษรของข้อความหลัง แท็ก <h1> ตรงนี้ให้เหมือน
    //เดิมหลังจากทำการแสดงแสดงผล

}

```

ตัวอย่างซอร์สโค้ดของการจัดการแท็ก <a> เพื่อมาแสดงผล

แท็ก <a> เป็นแท็กที่ใช้ในการสร้างจุดลิงค์(anchor) ไปยังเอกสารอื่นซึ่งจะต้องมีการจัดการในรูปแบบที่พิเศษกว่า คือจะต้องรีเทิร์นค่าจุดที่ลิงค์มาให้ผู้ใช้สามารถกดเลือกตำแหน่งนั้นๆได้

```

if (parserz.output_tag[Current_tag_value_attribute]=="<a href>")
{
    //Flag for position of begin and end of link
    // เป็นการจัดการเกี่ยวกับตำแหน่งที่จะลิงค์ไป
    String str=parserz.output_value[Current_tag_value_attribute];
    Link.AddLink(parserz.output_value[Current_tag_value_attribute]);
    Link.AddStartPt(CurrentPoint);
    Link.AddEndPt(str.length());
    Link.AddLine(CurrentLine);
    Link.next();

    // ทำการเปลี่ยนแปลงสีก่อน แสดง
    RGB.r=0;
    RGB.g=0;
    RGB.b=255;

    //เป็นการขีดเส้นใต้ให้กับข้อความที่จะลิงค์ไป
    g.setFont (Font.getFont (Font.FACE_SYSTEM, Font.STYLE_UNDERL
    INED, Font.SIZE_MEDIUM) );

    Display(parserz.output_value[Current_tag_value_attribute],g,RGB);

    //ทำการ รีเซ็ตให้เหมือนเดิม
    g.setFont (Font.getFont (Font.FACE_SYSTEM, Font.STYLE_PLAIN,
    Font.SIZE_MEDIUM) );

}

```

อธิบายเพิ่มเติมในส่วนของการแท็ก <a>

Link.AddStartPt(CurrentPoint); คือจุดพิกเซลเริ่มต้นของลิงค์ที่สามารถกดเลือกได้

Link.AddEndPt(str.length()); คือจุดพิกเซลสุดท้ายของลิงค์ที่สามารถกดเลือกได้

ตัวอย่างซอร์สโค้ดของการจัดการแท็ก <p>

ในแท็ก <p>จะมีความพิเศษตรงที่เมื่อกลุ่มของข้อความใดถูกแท็กนี้ครอบไว้ก็จะต้องมีการขึ้นบรรทัดใหม่โดยอัตโนมัติ ซึ่งจะต้องมีการกำหนดจุดขึ้นบรรทัดใหม่ไว้ด้วย

```

if (parserz.output_tag[Current_tag_value_attribute]=="<p>")
{
    //กำหนดให้ขึ้นบรรทัดใหม่และชิดซ้าย
    CurrentPoint=1;
    CurrentLine+=goto_NextLine;

    Display(parserz.output_value[Current_tag_value_attribute],g,RGB);
}

```

ตัวอย่างซอร์สโค้ดของการจัดการแท็ก

ในแท็ก จะมีความพิเศษจะมีแอตทริบิวต์ของรูปด้วย ซึ่งในขั้นตอนของการพาร์สนั้น จะเป็นการส่งค่าแอตทริบิวต์ทั้งหมดมาให้ส่วนแสดงผลเลย ดังนั้นในส่วนการแสดงผลเราจำเป็นต้องทำการแยกค่า แอตทริบิวต์แต่ละตัวอีกครั้งหนึ่ง

```

if(parserz.output_tag[Current_tag_value_attribute]=="<img>")
{
    //หากเจอแท็ก <img> ก็ให้ทำการเรียก ฟังก์ชัน Image() เพื่อทำการแยกแอตทริบิวต์
    //และ ประมวลผลต่อไป
    Image(parserz.output_tag_attr[Current_tag_value_attribute]);
}

```

ตัวอย่างซอร์สโค้ดของการจัดการแท็ก

ในแท็ก จะมีความพิเศษคือจะเป็นแท็กที่ใช้เพื่อทำการแสดงลิสต์ ซึ่งแท็กนี้จะเป็นแท็กร้อยของแท็ก และ อีกทีหนึ่ง ดังนั้นเราต้องมีการตรวจสอบก่อนการสั่งให้แสดงผลก่อนว่ามาจากแท็กร้อยใด ดังตัวอย่างโค้ดข้างล่างนี้

```

// List

if(parserz.output_tag[Current_tag_value_attribute]=="<li>")
{
    CurrentPoint=1;

    //    CurrentLine+=goto_NextLine;
    //เนื่องจากแท็กนี้ เป็นแท็กย่อย ของแท็ก <ul> และ <ol> ดังนั้นเราต้องทำการตรวจสอบก่อนว่า
    //แท็กย่อยที่รับมานั้น เป็นของแท็กอะไร

// Unorder List
// ตรวจสอบว่าเป็นของ unordered list หรือ ไม่หากใช้ก็ให้แสดงผลดัง โค้ดด้านล่างนี้
if (unorderedlist)
{

    g.fillRect(CurrentPoint,CurrentLine+6,4,4);

    CurrentPoint =CurrentPoint+7;

    parserz.output_value[Current_tag_value_attribute]=parserz.output_value[Current_tag_value_at
tribute];

    Display(parserz.output_value[Current_tag_value_attribute],g,RGB);

}

//End Unorder List

```

4.8 การสร้างส่วนสนับสนุนการโปรแกรมได้

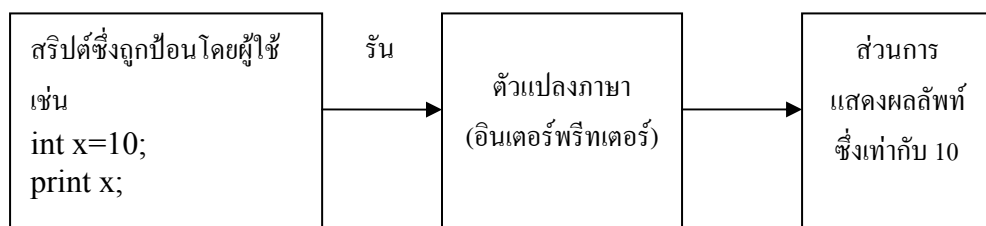
เนื่องจากบราวเซอร์ที่สามารถโปรแกรมได้นั้นจะต้องมีส่วนที่เป็นโครงสร้างของบราวเซอร์และส่วนที่เป็นส่วนอินเทอร์พรีเตอร์ ซึ่งส่วนของอินเทอร์พรีเตอร์จะต้องมีการใช้ทรัพยากรและโครงสร้างของบราวเซอร์ ดังนั้นจึงต้องมีการเตรียมส่วนสนับสนุนการทำงานของอินเทอร์พรีเตอร์ซึ่งประกอบด้วย

- 1) ส่วนสำหรับการรับข้อมูลและติดต่อกับผู้ใช้
- 2) ส่วนการจัดการหน่วยความจำ ซึ่งในการเขียนโปรแกรมบนโทรศัพท์เคลื่อนที่นั้นต้องเก็บบนเรคคอร์ดสตอร์

แนวคิดในการออกแบบ

ส่วนติดต่อกับผู้ใช้งานจะต้องประกอบด้วย

- 1) ส่วนการแก้ไขสคริปต์ (SCRIPT EDITOR) ซึ่งจะเป็นส่วนให้ผู้ใช้สามารถใส่ข้อความและโค้ดที่ใช้ในการโปรแกรม และสามารถเลือกที่เซฟไฟล์ แก้ไขไฟล์ โหลดไฟล์ ได้อีกด้วย ซึ่งจะทำให้การใช้งานมีความสะดวกมากขึ้น
- 2) ส่วนการรันสคริปต์ ซึ่งจะนำโค้ดที่พิมพ์ลงไปมาผ่านการทำงานของอินเทอร์พรีเตอร์ ซึ่งจะแปลงตัวโค้ดให้อยู่ในของคำสั่งที่สามารถทำงานได้จริง
- 3) ส่วนการแสดงผลลัพธ์ (CONSOLE) ซึ่งจะเป็นส่วนที่แสดงผลลัพธ์ของโค้ด



รูปที่ 4.8 ขั้นตอนการทำงานของส่วนติดต่อกับผู้ใช้งาน

ส่วนในการจัดการหน่วยความจำมีโครงสร้างดังต่อไปนี้



รูปที่ 4.9 การจัดการหน่วยความจำ

4.9 การเตรียมส่วนเก็บข้อมูล

ส่วนเก็บข้อมูลถาวร(RECORD STORE) จะช่วยอำนวยความสะดวกในการเก็บข้อมูลในการทำงานไว้แบบถาวร ซึ่งหมายถึง แม้จะปิดโปรแกรมลงไปแล้วก็สามารถที่จะเก็บข้อมูลเพื่อไว้ใช้ในการทำงานเมื่อเปิดโปรแกรมครั้งต่อไปได้ ซึ่งสามารถนำมาประยุกต์ในงานต่างๆดังต่อไปนี้

- ข้อมูลของผู้ใช้งานได้แก่
 - ยูอาร์แอลในปัจจุบัน ซึ่งจะเก็บยูอาร์แอลที่ผู้ใช้งานใช้งานอยู่ในปัจจุบัน ซึ่งโปรแกรมใช้ยูอาร์แอลตรงนี้เพื่อการแสดงผลเสมอ
 - ยูอาร์แอลของหน้าโฮมเพจ เพื่อแสดงหน้าแรกเมื่อเปิดบราวเซอร์ขึ้นมาใช้งาน

- ข้อมูลของรูปแบบการแสดงผลในปัจจุบัน ซึ่งจะทำให้ผู้ใช้สามารถปรับแต่งกราฟิกในการใช้งานได้
- ข้อมูลของสีของบราวเซอร์ ที่จะช่วยให้ผู้ใช้ปรับแต่งสีของบราวเซอร์ได้

#1 ยูอาร์แอลปัจจุบัน เช่น HTTP://WWW.CE.KMITL.AC.TH
#2 ยูอาร์แอลหน้าโฮมเพจ(หน้าแรก) เช่น HTTP://WWW.KMITL.AC.TH
#3 สไตล์ของบราวเซอร์ เช่น STANDARD
#4 สีของบราวเซอร์ เช่น GREEN
#5 สคริปต์ที่ใช้ในการทำงานปัจจุบัน

ตัวอย่างข้อมูลของผู้ใช้งานซึ่งอยู่ DATA STORE ชื่อ DB_1

- ข้อมูลสำหรับเก็บบุ๊กมาร์ค(BOOKMARK)
เมื่อผู้ใช้บราวเซอร์อยากจะบันทึกยูอาร์แอลหน้าเว็บเพจที่ตนเองชอบไว้ สามารถบันทึกเก็บไว้ในส่วน DATASTORE ของบุ๊กมาร์คไว้ เพื่อโหลดในครั้งหน้าได้อย่างสะดวก ซึ่งในส่วนนี้จะต้องมีการเก็บค่าไว้ของค่าต่อเรคคอร์ดคือ ส่วนชื่อของหน้าเว็บเพจนั้น และ ส่วนของยูอาร์แอลของหน้าเว็บเพจนั้นๆ

ชื่อ	ยูอาร์แอล
#1 KMITL	HTTP://WWW.KMITL.AC.TH
#2 YAHOO	HTTP://WWW.YAHOO.COM
#3 CE	HTTP://WWW.CE.KMITL.AC.TH
#4	

ตัวอย่างข้อมูลที่ BOOKMARK เก็บไว้ในส่วน DATA STORE ชื่อ DB_2

- ส่วนเก็บข้อมูลของการใช้งานเดินหน้าถอยหลัง(NEXT/BACK)
บราวเซอร์จะมีส่วนที่สามารถให้ผู้ใช้งานเลือกที่จะถอยหลังหน้าเพจ (BACK) และเลือกไปข้างหน้า เพจ (NEXT) ซึ่งจะทำให้สะดวกกว่าการป้อนยูอาร์แอลลงไปตรงๆ ซึ่งจะต้องใช้อินเด็กซ์เพื่อเป็นตัวชี้ตำแหน่งปัจจุบัน ซึ่งจะทำให้สามารถเดินหน้าถอยหลังได้อย่างถูกต้อง

#1 HTTP://WWW.KMITL.AC.TH	index
#2 HTTP://WWW.YAHOO.COM	

- ส่วนเก็บข้อมูลของการใช้งานสคริปต์สำหรับการโปรแกรม(Script)
บรรทัดต่อไปจะมีส่วนในการเก็บสคริปต์ ที่ผู้ใช้งานสามารถเก็บสคริปต์เพื่อเรียกขึ้นมาใช้
งาน ภายหลังได้

#1 INT I=10; PRINT(I);
#2 EVENTURL();
#3 A=B;E=D;
#4

4.10 ขั้นตอนการทดสอบบนโทรศัพท์เคลื่อนที่

เป็นขั้นตอนนำรับการนำโปรแกรมทั้งหมดไปทดสอบใช้งานจริงบนโทรศัพท์เคลื่อนที่ ซึ่งสิ่งที่ต้องคำนึงถึงหลักๆมี 2 อย่างคือ

- 1) ขนาดของโปรแกรม ซึ่งไม่ควรมีขนาดใหญ่เกินไป
- 2) การเชื่อมต่อโทรศัพท์กับเครือข่าย ซึ่งการเชื่อมต่อทดลองควรจะใช้การเชื่อมตรงกับเครื่องคอมพิวเตอร์ผ่านแท่นซิงค์(SYN PORT) ซึ่งจะมีความเร็วสูงและประหยัดกว่าการเชื่อมต่อผ่านจีพีอาร์เอส (GPRS) ซึ่งช้ากว่าและเสียค่าบริการ

ในส่วนการทำการทดสอบนั้นมีลำดับขั้นตอนการทำงานดังนี้

- 1) เตรียมไฟล์โปรแกรมมีดัดแปลงให้อยู่ในรูปของไฟล์นามสกุล *.JAR และ *.JAD
- 2) ติดตั้งซอฟต์แวร์สำหรับการส่งข้อมูลให้โทรศัพท์ ซึ่งจะมีมาพร้อมกับชุดของโทรศัพท์เรียบร้อยแล้ว
- 3) นำสายซิงค์เสียบเข้ากับพอร์ตยูเอสบีของเครื่องคอมพิวเตอร์
- 4) ใช้ซอฟต์แวร์ที่ติดตั้งเป็นตัวจัดการการนำโปรแกรมส่งไปไว้บนโทรศัพท์
- 5) ทดสอบใช้งานโปรแกรม ซึ่งจะสามารถทำงานได้ใกล้เคียงกับการทดลองภายในอิมูเลเตอร์ ซึ่งอาจแตกต่างกันบ้างในการแสดงผล รายละเอียดต่างๆ